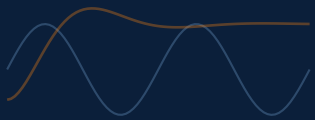
WEEK 2 · DAY 1 / 5

Mechatronics and Robotics Engineering (50 h)



Resonance in Robot Joints, Robot Anatomy and Forward Kinematics

3.7 Resonance & notch filters ▪ 4.1 Robot anatomy ▪ 4.2 Spatial descriptions ▪ 4.3 Forward kinematics

Mechatronics and Robotics Engineering Programme

Week 2 · Day 1 ▪ 2 weeks · 10 days · 50 hours

Agenda and learning outcomes

Code	Session	Time
3.7	Resonance, compliance and notch filters in robot joints	60 min
4.1	Robot anatomy and classification	60 min
4.2	Spatial descriptions: positions, rotations, homogeneous transforms	90 min
4.3	Forward kinematics: DH convention, planar 2-link, SCARA	90 min
	Total (+ 10-min breaks about every hour)	**5.0 h**

Today we leave the single motor axis and look at the **whole robot**: first the springy joint, then the arm as a chain of links and frames.

By the end of today you can . . .

- calculate the resonance frequency of a compliant joint
- explain what a notch filter does and when to use it
- name joint types, count DOF and classify robots
- build and use rotation matrices and homogeneous transforms
- write a DH table and compute the tool position of a 2-link arm and a SCARA

What we learned in Week 1 (1/2): actuators and control

Actuators – W1D1 and W1D2

- Motors turn current into torque: $T = k_t\, i$.
- Gearboxes trade speed for torque; the load "feels" J_L/N^2 to the motor.
- Harmonic and cycloidal gears in robot joints: high ratio, little backlash, **finite stiffness**.
- Size a motor with peak and RMS torque.

Control – W1D2 to W1D4

- Model $\rightarrow$ transfer function $\rightarrow$ poles.
- Feedback with PID; Bode plots, margins, bandwidth.
- Filters (LPF, notch, lead/lag) and digital control.
- Robot joints: independent-joint control, gravity compensation.

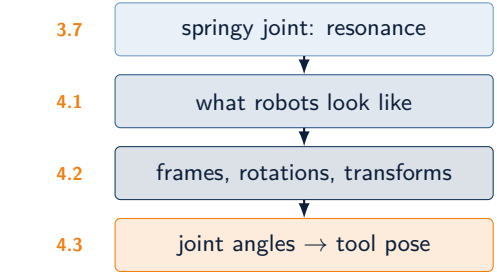


What we learned in Week 1 (2/2): servo drives, and today's step

Servo systems – W1D4 and W1D5

- Inverter + PWM make the motor voltage.
- FOC: torque comes from i_q.
- Cascade: current $\subset$ velocity $\subset$ position; tune inside out.
- Feedforward from the trajectory removes most of the following error.

Today: from one axis to a whole arm



Session 1

3.7 Resonance & notch filters

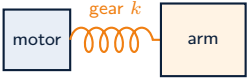
Why robot joints are springy

- W1D5 assumed a **rigid** load: motor and arm move as one body.
- Real joints are **compliant** (springy): the harmonic-drive flexspline, belts, shafts, even the arm links bend a little.
- Inertia + spring + little damping = a system that can **ring**.
- Raise the servo gains and, at some point, the arm buzzes or shakes.

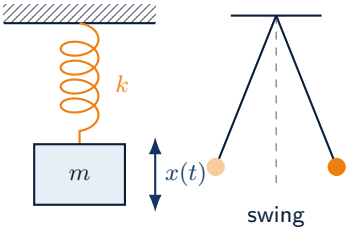
> Analogy: a fishing rod. Move the handle slowly and the tip follows; flick it fast and the tip whips back and forth.



Harmonic Drive strain-wave gear set. Pieceofmetalwork, CC BY-SA 4.0, via Wikimedia Commons



Refresher: natural frequency of a mass on a spring



- Pull and let go: it oscillates at its **natural frequency**.
- Push a swing in rhythm and it builds up: **resonance**.

$$\omega_n = \sqrt{\frac{k}{m}} \ \text{[rad/s]}, \qquad f_n = \frac{\omega_n}{2\pi} \ \text{[Hz]}$$

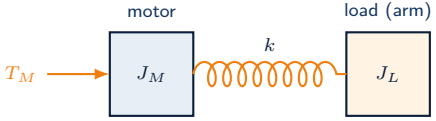
k stiffness [N/m], m mass [kg]. Rotating joint: k in N m/rad, J in kg m^2.

Example

1. $m = 1$ kg, $k = 100$ N/m
2. $\omega_n = \sqrt{100/1} = 10$ rad/s
3. $f_n = 10/(2\pi) \approx 1.6$ Hz

Stiffer $\Rightarrow$ higher f_n; heavier $\Rightarrow$ lower f_n.

The two-mass model and its two special frequencies



$$\omega_{\mathrm{AR}} = \sqrt{\frac{k}{J_L}}, \qquad \omega_{\mathrm{R}} = \omega_{\mathrm{AR}}\sqrt{1+R}$$

$$R = J_L/J_M \ \text{(inertia ratio)}$$

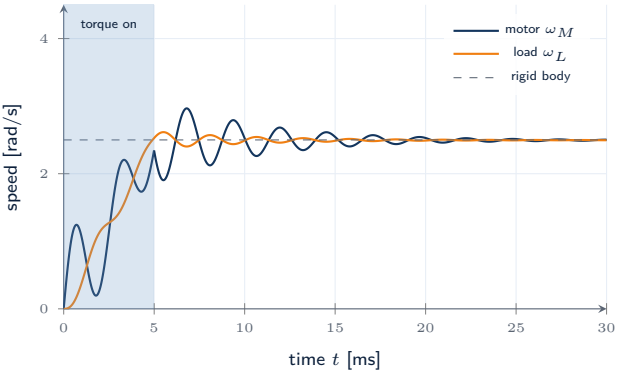
- **Anti-resonance** (AR): the load swings, the motor stands still.
- **Resonance** (R): motor and load swing against each other.

Worked example

1. $J_M = 1.2 \times 10^{-4}\,\mathrm{kg\,m^2}$, $J_L = 6 \times 10^{-4}\,\mathrm{kg\,m^2}$, $k = 600\,\mathrm{N\,m/rad}$
2. $\omega_{\mathrm{AR}} = \sqrt{600/(6 \cdot 10^{-4})} = 1000\,\mathrm{rad/s} \Rightarrow f_{\mathrm{AR}} = 159\,\mathrm{Hz}$
3. $R = 6/1.2 = 5$, $\sqrt{1+R} = 2.45$
4. $\omega_{\mathrm{R}} = 2449\,\mathrm{rad/s} \Rightarrow f_{\mathrm{R}} = 390\,\mathrm{Hz}$

Going further: Newton's law for each inertia with shaft torque $T_S = k(\theta_M - \theta_L) + c(\omega_M - \omega_L)$ gives the full transfer functions; through a gear N the load appears as J_L/N^2, k/N^2.

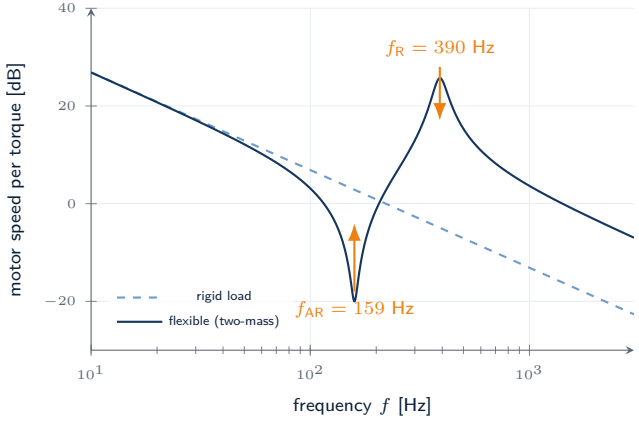
What resonance looks like in time: a short torque push



- Same two-mass system: 0.36 N m for 5 ms.
- A rigid body would just ramp up to 2.5 rad/s and stay there.
- Here motor and load **swap energy** through the spring at $f_R = 390$ Hz (period 2.6 ms).
- Low damping $\Rightarrow$ the ringing dies out slowly.

The encoder sits on the motor: the controller sees the blue curve, but the **tool** follows the orange one.

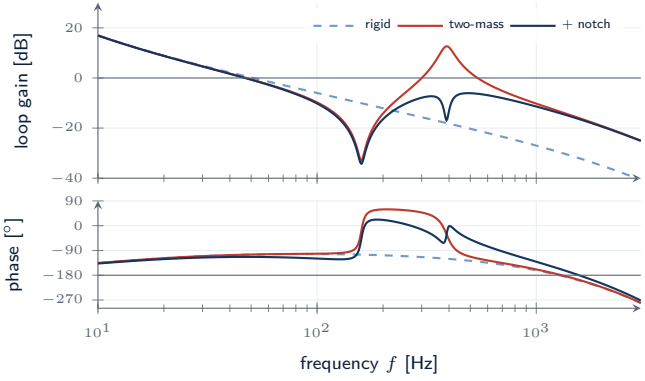
Frequency response of the two-mass system



- Low f: motor and load move as one body.
- Dip at f_{AR}, sharp **peak** at f_R.
- Above f_R only the motor moves: gain $1 + R = 6\times$ (≈ 15.6 dB) higher.
- Large inertia ratio $R \Rightarrow$ bigger jump, harder tuning.

Refresher: dB $= 20 \log_{10} |G|$; $+20$ dB $= \times 10$, $+6$ dB $\approx \times 2$.

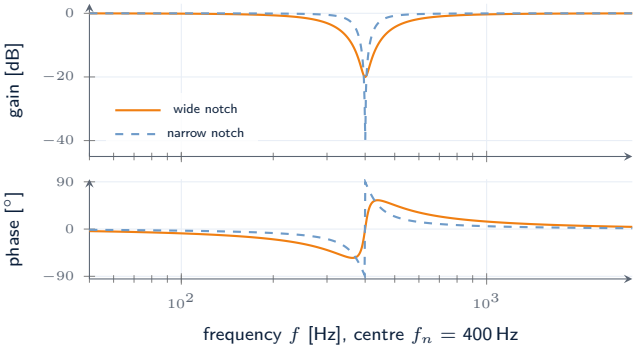
What the velocity loop sees



Velocity loop tuned for 50 Hz bandwidth, load from the example.

- Red: the resonance lifts the loop gain **above 0 dB** at 390 Hz ⇒ the axis rings.
- Blue: a **notch filter** at 390 Hz pushes the peak back down below 0 dB.
- Price: a little phase lost at 50 Hz (about $9°$).

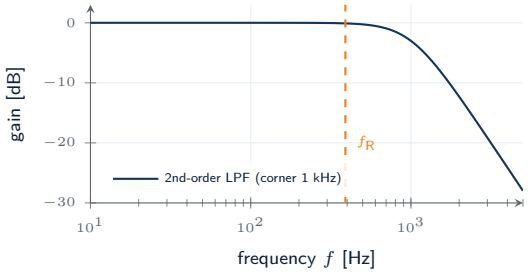
The notch filter: cutting out one frequency



- A **notch** (band-stop) filter removes a narrow band around f_n and passes everything else at 0 dB.
- It sits in the torque (current) command, after the velocity controller.
- **Depth**: how much it cuts. **Width**: how many Hz it covers.
- Wide = robust if f_R drifts, but costs more phase at low f.

Going further: biquad $N(s) = \dfrac{s^2 + 2\zeta_z\omega_n s + \omega_n^2}{s^2 + 2\zeta_p\omega_n s + \omega_n^2}$, depth ζ_z/ζ_p; Tustin discretisation (W1D4).

Notch or low-pass? Setting up the filter



A

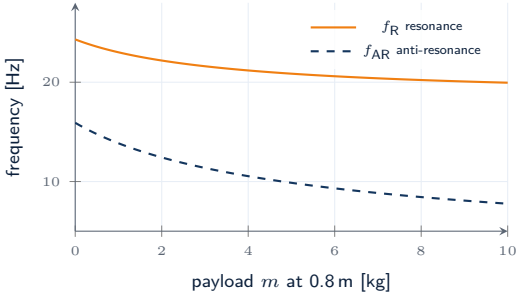
low-pass filter (LPF) is good against **broad** high-frequency noise, but at 390 Hz a 1 kHz LPF cuts almost nothing.

1. Measure the frequency response at low gain (sine sweep / chirp).
2. Read f_R from the peak; check it at several arm poses and payloads.
3. Set $f_n = f_R$, depth 10 dB to 20 dB.
4. Widen it if f_R moves by more than about $\pm 10\,\%$.
5. Raise the velocity gain again and re-check the margins.

Rule of thumb

Keep $f_n \gtrsim (3\text{–}5)\, f_c$. If the resonance is too close to the loop bandwidth, no filter helps: stiffen the mechanics or lower f_c.

Robot joints: the resonance moves with pose and payload



Joint with harmonic gear $N = 100$, joint stiffness 2×10^4 N m/rad.

Worked example: robot shoulder

1. Motor $J_m = 1.5 \times 10^{-4}$ kg m^2 seen at the joint: $J_m N^2 = 1.5$ kg m^2
2. Arm $J_L = 2$ kg m^2, $k = 2 \times 10^4$ N m/rad
3. $f_{AR} = \frac{1}{2\pi}\sqrt{2 \cdot 10^4 / 2} = 15.9$ Hz
4. $R = 2/1.5 = 1.33$, $f_R = 15.9\sqrt{2.33} = 24.3$ Hz

- Robot resonances are **low** (5 Hz to 30 Hz) and change with reach and payload.
- Hence: robust (wide) notches, gain scheduling, input shaping.

Check your understanding – 3.7

Check your understanding

1. The load inertia J_L is made 4 times larger. What happens to f_{AR}?
2. The velocity-loop bandwidth is 50 Hz, the resonance is at 390 Hz. Why a notch and not a low-pass filter?
3. The robot picks up a heavy part. Does its joint resonance go up or down?

Discuss with your neighbour for 2 minutes.

Summary 3.7 – Resonance and notch filters

Key take-aways

- A compliant joint = motor inertia + spring + load inertia.
- $f_{\mathrm{AR}} = \frac{1}{2\pi}\sqrt{k/J_L}$, $f_{\mathrm{R}} = f_{\mathrm{AR}}\sqrt{1+R}$.
- The resonance peak can lift the loop gain above 0 dB $\Rightarrow$ ringing.
- **Notch** for one narrow peak, **LPF** for broad noise.
- In robots f_{R} changes with pose and payload: fix the mechanics first, filter second.

Key terms

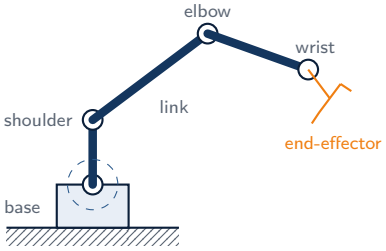
compliance springiness of a part
natural frequency rate of free oscillation
anti-resonance dip: motor still
resonance peak: masses swing apart
inertia ratio $R = J_L/J_M$
notch narrow band-stop filter

[Ellis, 2012]; [Spong et al., 2020, ch. 6].

Session 2

4.1 Robot anatomy

Anatomy of a robot arm



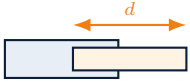
- **Links**: the rigid bodies ("bones").
- **Joints**: connect links and allow motion ("body joints"); each has a motor.
- **Base**: fixed to the floor; **wrist**: orients the tool.
- **End-effector** (EOAT): gripper, welding torch, suction cup (W1D2).
- Serial chain: base – link – joint – link – . . . – tool.

> Analogy: your own arm. Shoulder (3 rotations), elbow (1), wrist (3): about 7 joints.

Joint types: revolute and prismatic



Revolute (R)
rotation, variable θ [rad]



Prismatic (P)
sliding, variable d [m]

Both have **1 degree of freedom**. Others (spherical = 3, universal = 2) are made from R joints.

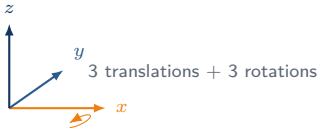
Joint variable

The one number that says where a joint is: angle θ_i (R) or stroke d_i (P). All of them together form the **joint vector** $\boldsymbol{q} = (q_1, \ldots, q_n)$.

- Robots are named by their joint string, base to wrist: e.g. **RRP** for a SCARA's first three, **PPP** for a gantry.
- The motor and encoder of W1 sit in every joint.

Degrees of freedom (DOF)

- A free body in space has **6 DOF**: 3 positions (x, y, z) + 3 orientations (roll, pitch, yaw).
- In a plane: 3 DOF (x, y, φ).
- To place a tool at *any* position *and* orientation, a robot needs **6 joints**.
- 7+ joints: **redundant** (like your arm): it can move the elbow while the hand stays still.



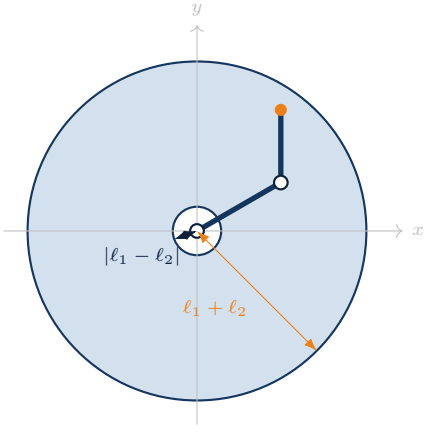
Counting mobility (Grübler)

Planar: $M = 3(n-1) - 2j$, n links incl. ground, j one-DOF joints.

Worked example

1. Planar 2-link arm: $n = 3$, $j = 2$
   $M = 3 \cdot 2 - 2 \cdot 2 = 2$
2. Four-bar linkage: $n = 4$, $j = 4$
   $M = 3 \cdot 3 - 2 \cdot 4 = 1$
3. Spatial 6-axis arm: $M = 6 \cdot 6 - 5 \cdot 6 = 6$

Workspace: where can the tool reach?



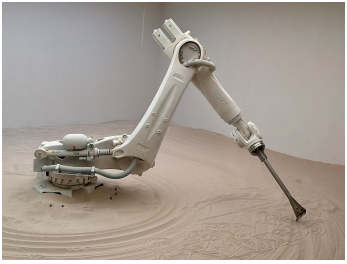
- **Workspace**: the set of all points the tool can reach.
- **Dexterous workspace**: points reachable with *any* orientation (smaller).
- Planar 2-link with full joint range: a ring between $|\ell_1 - \ell_2|$ and $\ell_1 + \ell_2$.

Example

$\ell_1 = 0.4\,\text{m}$, $\ell_2 = 0.3\,\text{m}$: inner radius 0.1 m, outer 0.7 m. A part at 0.75 m is **out of reach**; move the robot or the part.

Real joints have limits (cables, collisions), so the true workspace is smaller than the ring.

Articulated arms and SCARA robots



KUKA industrial arm. Oleg Yunakov, CC BY-SA 4.0, via Wikimedia Commons

- **Articulated** (6R): like a human arm; large spherical workspace.
- Welding, painting, machine tending, palletising.



KUKA KR10 SCARA. Jo Teichmann, Public domain, via Wikimedia Commons

- **SCARA** (RRPR): two vertical-axis R joints, one vertical P, one tool rotation.
- Stiff vertically, compliant sideways: fast pick-and-place, assembly.

Cartesian / gantry and delta (parallel) robots



Gantry robot over a palletising line. www.tecnowey.com, CC BY 3.0, via Wikimedia Commons

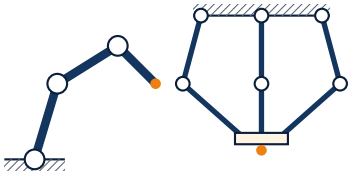
- **Cartesian / gantry** (PPP): three linear axes, box-shaped workspace.
- Simple kinematics, scalable size: 3D printers, CNC, large handling.



Delta X robot. Trungdoanhong, CC BY-SA 4.0, via Wikimedia Commons

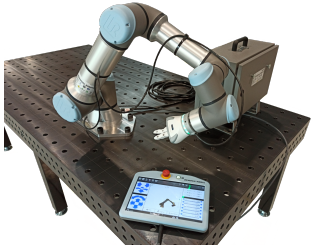
- **Delta** (parallel): three arms share one light platform.
- Motors stay on the base ⇒ very fast picking (food, pharma).

Serial vs. parallel kinematic chains



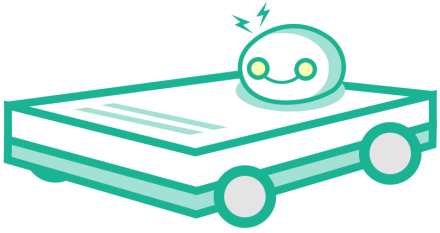
	Serial	Parallel
Chain	open, one path	closed loops
Workspace	large	small
Stiffness	lower (errors add)	high
Moving mass	motors ride along	motors on base
Speed	medium	very high
Kinematics	forward easy	inverse easy

Collaborative and mobile robots



UR16e cobot with teach pendant. Auledas, CC BY-SA 4.0, via Wikimedia Commons

- **Cobot**: light 6–7 axis arm made to work **next to people**.
- Joint torque sensing, speed and force limits (W2D4: ISO/TS 15066).



Autonomous mobile robot (illustration). Rlistmedia, CC BY 4.0, via Wikimedia Commons

- **Mobile robot** (AGV / AMR): wheels instead of a fixed base.
- Mobile manipulator = AMR + arm: the workspace travels.

Classification at a glance

Type	Joints	DOF	Workspace	Typical use
Articulated	RRRRRR	6	sphere-like	welding, painting, tending
SCARA	RRPR	4	cylinder	assembly, pick-and-place
Cartesian / gantry	PPP(R)	3–4	box	3D printing, palletising, CNC loading
Delta	3 parallel arms (+R)	3–4	small dome	high-speed picking
Cobot	6–7 R	6–7	sphere-like	shared work with people
Mobile (AMR)	2 driven wheels	3 (plane)	the floor	logistics, intralogistics

How to choose

Start from the task: **reach** and workspace shape, **payload**, **speed**/cycle time, **accuracy**, and whether people share the space (W2D3: work-cell design).

Check your understanding – 4.1

Check your understanding

1. How many joints does a robot need to place a tool at any position *and* orientation in space?
2. A planar arm has links $\ell_1 = 0.5\,\text{m}$ and $\ell_2 = 0.5\,\text{m}$. What is its workspace?
3. Why is a delta robot faster than an articulated arm of similar size?

Discuss with your neighbour for 2 minutes.

Summary 4.1 – Robot anatomy and classification

Key take-aways

- A robot = links + joints (R or P) + base + end-effector.
- 6 DOF to set position and orientation; more = redundant.
- Workspace follows from link lengths and joint limits.
- Serial: large workspace; parallel: stiff and fast.
- Choose by reach, payload, speed, accuracy, people.

Key terms

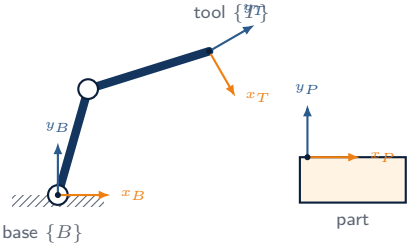
link / joint body / connection
revolute, prismatic rotate / slide
DOF independent motions
workspace reachable points
SCARA, delta, gantry robot types
cobot, AMR people-safe arm / mobile base

[Craig, 2005, ch. 1]; [Siciliano et al., 2009, ch. 1].

Session 3

4.2 Spatial descriptions

Why we need coordinate frames



- Every object gets its own **frame**: an origin + 3 perpendicular axes.
- **Base** frame $\{B\}$, **tool** frame $\{T\}$, part (user) frame $\{P\}$.
- The camera sees the part in *its* frame; the robot needs it in the base frame.
- Task: describe one frame **relative to another**.

> Analogy: "two metres left of the door" only makes sense if you know where the door is and which way you face.

Refresher: vectors and matrix–vector multiplication

Refresher: matrix times vector

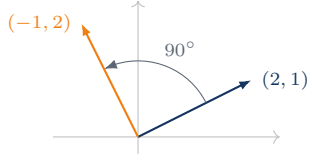
Each output row = row of the matrix "dotted" with the vector:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

- A position vector ${}^{A}\boldsymbol{p} = (p_x, p_y, p_z)^T$: the point, measured in frame $\{A\}$.
- The superscript says **which frame** we measure in.

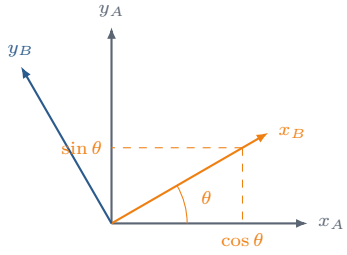
Example

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \cdot 2 + (-1) \cdot 1 \\ 1 \cdot 2 + 0 \cdot 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$



This matrix rotates any vector by $90°$.

Rotation in the plane



- Frame $\{B\}$ is turned by θ relative to $\{A\}$.
- Unit vector $\hat{x}_B$ seen in $\{A\}$: $(\cos\theta, \sin\theta)$; $\hat{y}_B$: $(-\sin\theta, \cos\theta)$.
- Put them side by side as **columns**:

$$ {}^{A}R_{B}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}, \qquad {}^{A}\boldsymbol{p} = {}^{A}R_{B}\,{}^{B}\boldsymbol{p} $$

Example

$\theta = 30^\circ$, ${}^{B}\boldsymbol{p} = (1, 0)$: ${}^{A}\boldsymbol{p} = (\cos 30^\circ, \sin 30^\circ) = (0.866,\ 0.5)$.

Properties of rotation matrices

- **Columns** = the axes of $\{B\}$ written in $\{A\}$.
- Each column has length 1, columns are perpendicular: the matrix is **orthonormal**.
- Inverse is just the transpose: ${}^{B}R_{A} = ({}^{A}R_{B})^{-1} = ({}^{A}R_{B})^{T}$.
- $\det R = +1$ (a right-handed frame stays right-handed).
- 9 numbers, but only 3 are free (3 rotational DOF).

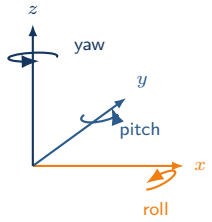
Worked check

$$R = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \ (90^\circ)$$

1. Column lengths: $\sqrt{0^2 + 1^2} = 1$ ✓
2. Dot product of columns: $0 \cdot (-1) + 1 \cdot 0 = 0$ ✓
3. $R^T R = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = I$ ✓

Turning back by $-\theta$ = transposing: $\sin(-\theta) = -\sin\theta$.

Rotations in 3D: the three elementary rotations



Right-hand rule: thumb along the axis, fingers show positive rotation.

$$R_z(\theta) = \begin{bmatrix} c_\theta & -s_\theta & 0 \\ s_\theta & c_\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\theta & -s_\theta \\ 0 & s_\theta & c_\theta \end{bmatrix} \quad R_y(\theta) = \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix}$$

$c_\theta = \cos\theta$, $s_\theta = \sin\theta$.

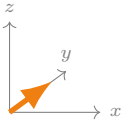
- R_z is the 2D rotation with z kept fixed.
- Note the sign swap in R_y (cyclic order $z \to x$).

Order matters: composing rotations

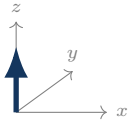
- Rotations combine by **multiplication**: ${}^{A}R_{C} = {}^{A}R_{B}\,{}^{B}R_{C}$.
- Matrix products are **not commutative**: $R_1 R_2 \neq R_2 R_1$.
- Try it with a book: turn about z then x, versus x then z.

Worked example: where does $\hat{x}$ go?

1. $R_z(90°)R_x(90°)\,\hat{x} = R_z(90°)\,\hat{x} = \hat{y}$
2. $R_x(90°)R_z(90°)\,\hat{x} = R_x(90°)\,\hat{y} = \hat{z}$
3. Different results $\Rightarrow$ order matters.



z then x

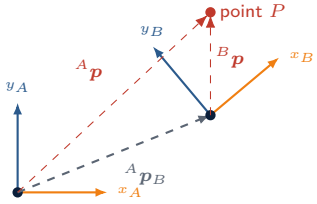


x then z

The thick arrow is the final direction of $\hat{x}$.

Going further: roll–pitch–yaw and **Euler angles** (e.g. ZYZ) describe any orientation with 3 angles; robot controllers often use **quaternions** (4 numbers) to avoid the "gimbal lock" singularity.

Rotation + translation: the full pose of a frame



- Pose of $\{B\}$ in $\{A\}$: **orientation** ${}^{A}R_B$ and **origin position** ${}^{A}\boldsymbol{p}_B$.
- A point known in $\{B\}$ is first rotated, then shifted:

$$ {}^{A}\boldsymbol{p} = {}^{A}R_B \, {}^{B}\boldsymbol{p} + {}^{A}\boldsymbol{p}_B $$

- Works, but chaining many frames gets messy: rotate, add, rotate, add ...
- Trick on the next slide: pack both into **one matrix**.

The homogeneous transform

$$\begin{bmatrix} {}^A\boldsymbol{p} \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} {}^A R_B & {}^A\boldsymbol{p}_B \\ \mathbf{0}^T & 1 \end{bmatrix}}_{{}^A T_B \ (4\times 4)} \begin{bmatrix} {}^B\boldsymbol{p} \\ 1 \end{bmatrix}$$

- Add a "1" to each point: rotation *and* shift become one multiplication.
- In the plane: a 3×3 matrix with R (2×2) and $\boldsymbol{p}$ (2×1).
- Chains are now just products: ${}^A T_C = {}^A T_B \, {}^B T_C$.

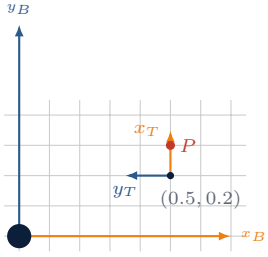
translation

$$\begin{bmatrix} r_{11} & r_{12} & r_{13} & p_x \\ r_{21} & r_{22} & r_{23} & p_y \\ r_{31} & r_{32} & r_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

rotation

"Homogeneous" = the extra 1. Industrial controllers store every frame (tool, user, base) as such a matrix or as $(x, y, z, 3$ angles$)$.

Worked example: a point on the tool, seen from the base



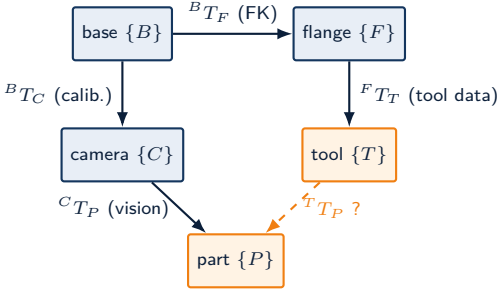
grid 0.1 m

Worked example

Tool frame $\{T\}$ at $(0.5, 0.2)$ m, turned $90°$; point P is 0.1 m along x_T.

1. ${}^{B}T_{T} = \begin{bmatrix} 0 & -1 & 0.5 \\ 1 & 0 & 0.2 \\ 0 & 0 & 1 \end{bmatrix}$, ${}^{T}\boldsymbol{p} = (0.1,\, 0,\, 1)^{T}$
2. Row 1: $0 \cdot 0.1 - 1 \cdot 0 + 0.5 = 0.5$
3. Row 2: $1 \cdot 0.1 + 0 \cdot 0 + 0.2 = 0.3$
4. ${}^{B}\boldsymbol{p} = (0.5,\, 0.3)$ m ✓ (matches the picture)

Chaining frames: base, tool, camera, part



- Follow arrows = multiply matrices; go against an arrow = use the inverse.
- Where is the part relative to the tool?

$$ {}^{T}T_{P} = ({}^{B}T_{F}\,{}^{F}T_{T})^{-1}\,{}^{B}T_{C}\,{}^{C}T_{P} $$

- This "transform graph" is used in every vision-guided pick (W2D5: hand–eye calibration).

Inverting a homogeneous transform

- ${}^{A}T_{B}$ says where $\{B\}$ is seen from $\{A\}$; the inverse says where $\{A\}$ is seen from $\{B\}$.
- No need for a general matrix inverse: use $R^{-1} = R^{T}$.

$$({}^{A}T_{B})^{-1} = {}^{B}T_{A} = \begin{bmatrix} R^{T} & -R^{T}\boldsymbol{p} \\ \mathbf{0}^{T} & 1 \end{bmatrix}$$

Worked example (tool from before)

1. $R = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$, $\boldsymbol{p} = \begin{bmatrix} 0.5 \\ 0.2 \end{bmatrix}$
2. $R^{T} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$
3. $-R^{T}\boldsymbol{p} = -\begin{bmatrix} 0.2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -0.2 \\ 0.5 \end{bmatrix}$
4. Check: base origin seen from the tool is 0.2 m behind x_T, 0.5 m along y_T ✓

Check your understanding – 4.2

Check your understanding

1. What do the columns of a rotation matrix ${}^{A}R_{B}$ tell you?
2. Why do we add a "1" to position vectors in homogeneous coordinates?
3. Is $R_x(90°)R_z(90°)$ equal to $R_z(90°)R_x(90°)$? Why does it matter for a robot wrist?

Discuss with your neighbour for 2 minutes.

Summary 4.2 – Spatial descriptions

Key take-aways

- Frames give every object an origin and axes.
- ${}^{A}R_{B}$: columns = axes of $\{B\}$ in $\{A\}$; $R^{-1} = R^{T}$.
- 3D: R_x, R_y, R_z; order of rotations matters.
- ${}^{A}T_{B}$ packs rotation and translation in one 4×4 matrix.
- Chains multiply; go backwards with $[R^{T},\ -R^{T}\boldsymbol{p}]$.

Key terms

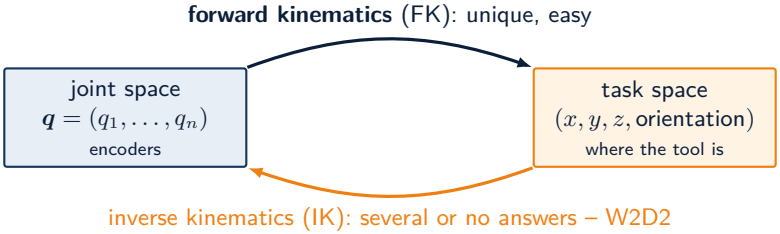
frame origin + 3 axes
pose position + orientation
rotation matrix orthonormal 3×3
homogeneous transform 4×4 pose
RPY / Euler 3-angle orientation
transform graph chain of frames

[Craig, 2005, ch. 2]; [Lynch & Park, 2017, ch. 3].

Session 4

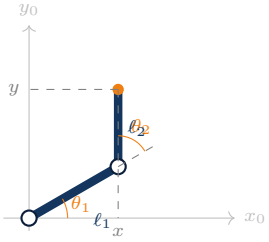
4.3 Forward kinematics

Forward kinematics: from joint angles to tool pose



- **Forward kinematics**: given all joint values, compute the tool frame ${}^{0}T_{n}(\boldsymbol{q})$.
- Pure geometry: no forces, no time.
- Used to display the tool position, check limits, and simulate (digital twin, W2D3).
- Recipe: one transform per joint, multiply them all.

The planar 2-link arm: geometry first



- Walk along the arm: first ℓ_1 at angle θ_1, then ℓ_2 at angle $\theta_1 + \theta_2$.
- θ_2 is measured **relative to link 1**.

$$x = \ell_1 \cos\theta_1 + \ell_2 \cos(\theta_1 + \theta_2)$$
$$y = \ell_1 \sin\theta_1 + \ell_2 \sin(\theta_1 + \theta_2)$$
$$\varphi = \theta_1 + \theta_2$$

x, y tool position [m], φ tool orientation, ℓ_i link lengths [m].

Worked example: 2-link forward kinematics

Worked example

Given $\ell_1 = 0.4\,\text{m}$, $\ell_2 = 0.3\,\text{m}$, $\theta_1 = 30^\circ$, $\theta_2 = 60^\circ$.

1. $\theta_1 + \theta_2 = 90^\circ$
2. $x = 0.4\cos 30^\circ + 0.3\cos 90^\circ = 0.346 + 0 = 0.346\,\text{m}$
3. $y = 0.4\sin 30^\circ + 0.3\sin 90^\circ = 0.2 + 0.3 = 0.5\,\text{m}$
4. $\varphi = 90^\circ$: the gripper points straight up.

Sanity checks: distance from base $\sqrt{0.346^2 + 0.5^2} = 0.61\,\text{m} \le \ell_1 + \ell_2 = 0.7\,\text{m}$ ✓ (same pose as the picture before).

- Units: angles in degrees or radians, but be consistent in your calculator!
- $\theta_2 = 0$: arm stretched, tool at $\ell_1 + \ell_2$.
- $\theta_2 = 180^\circ$: arm folded, tool at $|\ell_1 - \ell_2|$.

The same result as a product of transforms

Each joint = "rotate by θ_i, then move ℓ_i along the new x":

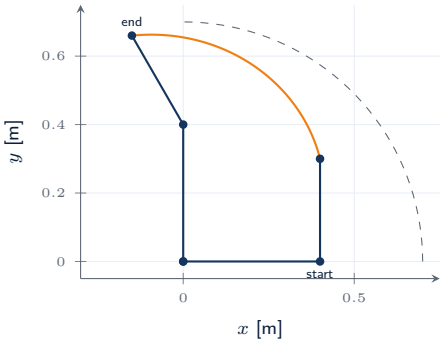
$$ {}^{i-1}T_i = \begin{bmatrix} c_i & -s_i & \ell_i c_i \\ s_i & c_i & \ell_i s_i \\ 0 & 0 & 1 \end{bmatrix} $$

$$ {}^{0}T_2 = {}^{0}T_1\,{}^{1}T_2 = \begin{bmatrix} c_{12} & -s_{12} & \ell_1 c_1 + \ell_2 c_{12} \\ s_{12} & c_{12} & \ell_1 s_1 + \ell_2 s_{12} \\ 0 & 0 & 1 \end{bmatrix} $$

$c_{12} = \cos(\theta_1 + \theta_2)$, $s_{12} = \sin(\theta_1 + \theta_2)$.

- The last column is the tool **position** – the same x, y as the geometry.
- The upper-left block is the tool **orientation** $\varphi = \theta_1 + \theta_2$.
- Adding a joint = multiplying one more matrix. This scales to 6 joints; geometry by hand does not.

Joint space vs. task space: a straight joint move is a curve



- Joints move linearly: $\theta_1 : 0 \to 90°$, $\theta_2 : 90° \to 30°$, at the same time.
- FK maps every intermediate $\boldsymbol{q}$ to the tool: an **arc**, not a straight line.
- This is a robot's "PTP" (joint) move; a "LIN" move needs IK at every step (W2D2, 4.7).

Dashed: outer workspace boundary $\ell_1 + \ell_2 = 0.7\,\text{m}$.

Why a convention? The Denavit–Hartenberg (DH) idea

- In 3D one could place frames anywhere: many numbers, many mistakes.
- **DH convention** (1955): put frames in a **standard way** so that each joint needs only **4 numbers**.
- One joint variable among them: θ_i (R) or d_i (P); the other 3 are fixed by the link geometry.
- Robot data sheets and simulators list a **DH table**.

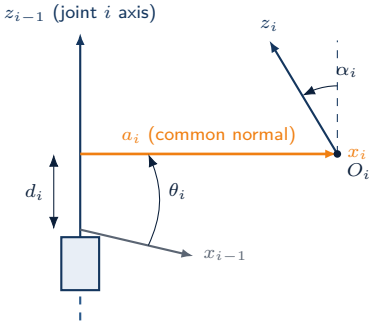
The 4 DH parameters of link i

θ_i	rotate about z_{i-1} (joint angle)
d_i	slide along z_{i-1} (offset)
a_i	slide along x_i (link length)
α_i	rotate about x_i (link twist)

$$A_i = R_z(\theta_i)\, T_z(d_i)\, T_x(a_i)\, R_x(\alpha_i)$$

Standard (distal) DH as in Spong; Craig uses a "modified" variant – same idea, different bookkeeping.

The four DH parameters in a picture



- z_{i-1} lies along joint i's axis of motion.
- x_i lies along the **common normal**: the shortest line between z_{i-1} and z_i.
- a_i, α_i: how far apart and how twisted the two joint axes are.
- d_i, θ_i: how far along and how turned about the previous axis.

Parallel joint axes (planar arms, SCARA): $\alpha_i = 0$ and the picture becomes flat.

DH recipe: a step-by-step procedure

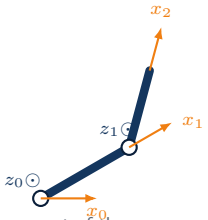
1. Number the joints $1 \dots n$ from the base; draw each joint axis.
2. Put z_{i-1} along the axis of joint i.
3. Put x_i along the common normal from z_{i-1} to z_i; y_i by the right-hand rule.
4. Read $\theta_i, d_i, a_i, \alpha_i$ for each link; mark the joint variable.
5. Multiply: ${}^0T_n = A_1 A_2 \cdots A_n$.

Tips for students

- Draw big, use 3 colours for x, y, z.
- Parallel axes: choose $d_i = 0$ where you can.
- Check one easy pose (all zeros) by eye.
- Add a fixed tool transform ${}^nT_{\text{tool}}$ at the end.

Going further: DH breaks down for exactly parallel axes with small misalignment (calibration); the Hayati modification or product-of-exponentials (Lynch & Park) is then used.

DH table of the planar 2-link arm

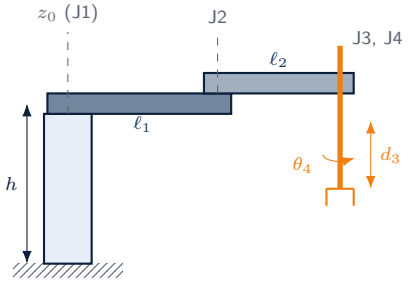


Link	θ_i	d_i	a_i	α_i
1	θ_1^*	0	ℓ_1	0
2	θ_2^*	0	ℓ_2	0

* joint variable

- $\alpha_i = 0$ and $d_i = 0$: A_i reduces to the 3×3 matrix we already used (in 4×4 form with z unchanged).
- ${}^0T_2 = A_1 A_2$ gives the same x, y, φ as the geometry ✓

The SCARA robot: structure



Side view: J1, J2, J4 rotate about vertical axes; J3 slides vertically.

- Joints: **R R P R** – 4 DOF: x, y, z and one rotation about z.
- All joint axes are **parallel** (vertical): the x–y part is exactly our 2-link arm!
- z comes only from the prismatic joint d_3.
- Ideal for placing parts on a table: screwing, inserting, packing.

Selective Compliance Assembly Robot Arm: stiff in z (pressing), slightly compliant in x–y (finding the hole).

SCARA: DH table and forward kinematics

Link	θ_i	d_i	a_i	α_i
1	θ_1^*	h	ℓ_1	0
2	θ_2^*	0	ℓ_2	180°
3	0	d_3^*	0	0
4	θ_4^*	0	0	0

$\alpha_2 = 180^\circ$ flips z to point down, so $d_3 > 0$ moves the tool down. [Spong et al., 2020]

$$x = \ell_1 c_1 + \ell_2 c_{12}, \quad y = \ell_1 s_1 + \ell_2 s_{12}$$
$$z = h - d_3, \qquad \varphi = \theta_1 + \theta_2 - \theta_4$$

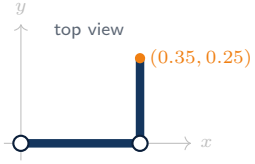
- Same x, y as the 2-link arm; z is decoupled.
- The minus sign on θ_4 comes from the flipped z axis.

Worked example: SCARA tool position

Worked example

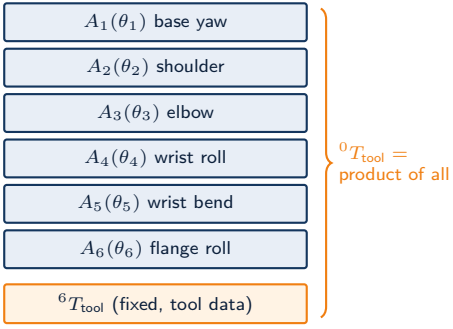
Given $h = 0.4\,\text{m}$, $\ell_1 = 0.35\,\text{m}$, $\ell_2 = 0.25\,\text{m}$; joints $\theta_1 = 0^\circ$, $\theta_2 = 90^\circ$, $d_3 = 0.1\,\text{m}$, $\theta_4 = 0^\circ$.

1. $x = 0.35 \cdot 1 + 0.25 \cos 90^\circ = 0.35\,\text{m}$
2. $y = 0.35 \cdot 0 + 0.25 \sin 90^\circ = 0.25\,\text{m}$
3. $z = 0.4 - 0.1 = 0.3\,\text{m}$
4. $\varphi = 0 + 90^\circ - 0 = 90^\circ$



Top view: the elbow is bent 90°; the gripper is 0.3 m above the floor.

Six-axis arms: same recipe, more matrices



- Typical industrial arm: first 3 joints place the wrist, last 3 (axes meeting in one point: **spherical wrist**) orient the tool.
- The controller evaluates this product every cycle (e.g. every 1 ms) to show and limit the tool pose.
- Accuracy depends on how well the DH numbers match the real robot $\Rightarrow$ **kinematic calibration**.

Going further: a full 6R DH table (e.g. for a PUMA-type arm) is in [Craig, 2005, ch. 3]; the spherical wrist makes the IK split into position and orientation parts (W2D2).

Check your understanding – 4.3

Check your understanding

1. A 2-link arm has $\ell_1 = \ell_2 = 0.5\,\text{m}$, $\theta_1 = 0°$, $\theta_2 = 90°$. Where is the tool?
2. How many DH parameters per link, and which one is the joint variable for a prismatic joint?
3. In a SCARA, which joint sets the height z? Does it change x or y?

Discuss with your neighbour for 2 minutes.

Summary 4.3 – Forward kinematics

Key take-aways

- FK: joint values $\rightarrow$ tool pose; unique and easy.
- 2-link: $x = \ell_1 c_1 + \ell_2 c_{12}$, $y = \ell_1 s_1 + \ell_2 s_{12}$.
- DH: 4 numbers per link, $A_i = R_z T_z T_x R_x$.
- ${}^0T_n = A_1 \cdots A_n$, then the fixed tool transform.
- SCARA = 2-link in x–y + prismatic z + tool roll.

Key terms

forward kinematics $\boldsymbol{q} \rightarrow$ pose
joint / task space $\boldsymbol{q}$ vs. $(x, y, z, \ldots)$
DH parameters θ, d, a, α
common normal line between axes
spherical wrist 3 axes meet
calibration matching model to robot

[Denavit & Hartenberg, 1955]; [Spong et al., 2020, ch. 3].

Session 5

W2D1 Wrap-up

Day summary

The joint and the robot

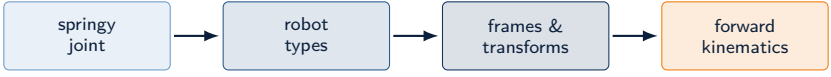
3.7 Compliant joints ring at $f_R = f_{AR}\sqrt{1+R}$; notch filters remove one peak; robot resonances move with pose and payload.

4.1 Links + R/P joints; 6 DOF for full pose; articulated, SCARA, gantry, delta, cobot, AMR.

The mathematics of position

4.2 Rotation matrices, homogeneous transforms ${}^{A}T_{B}$, chains and inverses.

4.3 Forward kinematics with DH: 2-link arm and SCARA worked out by hand.



Preview of W2D2

Robot Mechanisms II + Robotic Manufacturing I

- 4.4 Inverse kinematics: from a tool target back to joint angles (elbow up / down).
- 4.5 The Jacobian: joint speeds $\rightarrow$ tool speed; singularities; $\boldsymbol{\tau} = J^T \boldsymbol{F}$.
- 4.6 Robot dynamics intuition: gravity, inertia, payload.
- 4.7 Trajectories: PTP, linear and circular moves; AMR kinematics.
- 5.1 Flexible automation and Industry 4.0.

Bring along from today
– the 2-link FK formulas,
– ${}^{A}T_{B}$ and its inverse,
– the workspace ring $|\ell_1 - \ell_2| \ldots \ell_1 + \ell_2$.

Figure credits

All plots, block diagrams, robot sketches and frame drawings without a credit line are original drawings made for this course (data from `w2d1/data/gen_data.py`; two-mass, loop and filter data reused from the earlier course material). Downloaded figures (Wikimedia Commons):

File	Source (Wikimedia Commons) and author	Licence
d1_harmonic_drive.jpg	File:Harmonic Drive AG strain wave gear set.jpg – Pieceofmetalwork	CC BY-SA 4.0
d4_kuka_robot.jpg	File:KUKA industrial robotic arm performing "The Beach" by Roy Nachum - 02.jpg – Oleg Yunakov	CC BY-SA 4.0
r21_scara.jpg	File:KUKA Industrial Robot KR10 SCARA.jpg – Jo Teichmann	Public domain
r21_gantry.jpg	File:Robot Portico tecno-840.jpg – www.tecnowey.com	CC BY 3.0
r21_delta.jpg	File:Delta x robot.jpg – Trungdoanhong	CC BY-SA 4.0
r21_cobot.png	File:UR16e robot arm.png – Auledas	CC BY-SA 4.0
r21_mobile.png	File:Autonomous Mobile Robot AMR.png – Rlistmedia	CC BY 4.0

References

- Craig, J. J.: *Introduction to Robotics: Mechanics and Control*, 3rd ed. Pearson, 2005.
- Spong, M. W.; Hutchinson, S.; Vidyasagar, M.: *Robot Modeling and Control*, 2nd ed. Wiley, 2020.
- Siciliano, B.; Sciavicco, L.; Villani, L.; Oriolo, G.: *Robotics: Modelling, Planning and Control*. Springer, 2009.
- Lynch, K. M.; Park, F. C.: *Modern Robotics: Mechanics, Planning, and Control*. Cambridge Univ. Press, 2017.
- Corke, P.: *Robotics, Vision and Control*, 2nd ed. Springer, 2017.
- Denavit, J.; Hartenberg, R. S.: A kinematic notation for lower-pair mechanisms based on matrices. *ASME J. Appl. Mech.* 22, 1955.
- Siciliano, B.; Khatib, O. (eds.): *Springer Handbook of Robotics*, 2nd ed. Springer, 2016.
- Ellis, G.: *Control System Design Guide*, 4th ed. Butterworth-Heinemann, 2012.
- Franklin, G.; Powell, J. D.; Emami-Naeini, A.: *Feedback Control of Dynamic Systems*. Pearson.
- Åström, K. J.; Murray, R. M.: *Feedback Systems*. Princeton Univ. Press, 2008.
- Singer, N. C.; Seering, W. P.: Preshaping command inputs to reduce system vibration. *ASME J. Dyn. Sys. Meas. Control* 112(1), 1990.
- ISO 8373: Robotics – Vocabulary.
- ISO 9787: Robotics – Coordinate systems and motion nomenclatures.
- ISO 10218-1/-2: Robots and robotic devices – Safety requirements for industrial robots.